Fake Tool Adapters

This package contains deterministic fake adapters for Phase 1 eval runs.

The fake adapters are intentionally local and in-memory. They do not call real Notion, Google Calendar, Gmail, or system clipboard APIs. Their job is to let the eval harness execute tool calls against scripted scenario state before real integrations exist.

Shared Result

All fake adapters return `FakeToolResult` from `fake_base.py`.

Fields:

- `success`: whether the fake tool call succeeded
- `tool`: the tool name, such as `calendar.create_event`
- `error`: failure message when `success` is false
- `data`: optional structured output, such as a generated ID or clipboard text

Calendar

Module:

```
baymax.tools.fake_calendar
```

State key:

```
calendar_events
```

Supported tools:

- `calendar.create_event`
- `calendar.update_event`

Notes:

- events are validated with `CalendarEvent`
- generated event IDs use `evt_fake_calendar_###`
- generated IDs skip existing fixture IDs
- updates re-validate the full event after changes

Notion

Module:

```
baymax.tools.fake_notion
```

State key:

```
notion_tasks
```

Supported tools:

- `notion.create_task`
- `notion.update_task`

Notes:

- tasks are validated with `NotionTask`
- task status is currently `open` or `done`
- generated task IDs use `task_fake_notion_###`
- generated IDs skip existing fixture IDs
- updates re-validate the full task after changes

Gmail

Module:

```
baymax.tools.fake_gmail
```

State keys:

```
gmail_contacts
gmail_messages
gmail_drafts
sent_emails
```

Supported tools:

- `gmail.create_draft`
- `gmail.send_email`

Notes:

- contacts, messages, drafts, and sent emails are validated with Pydantic models
- draft creation and email sending are separate operations
- `gmail.create_draft` does not add to `sent_emails`

- `gmail.send_email` does not add to `gmail_drafts`
- generated draft IDs use `draft_fake_gmail_###`
- generated sent-email IDs use `email_fake_gmail_###`
- generated IDs skip existing fixture IDs

System Clipboard

Module:

```
baymax.tools.fake_clipboard
```

State key:

```
clipboard
```

Supported tools:

- `clipboard.read`
- `clipboard.write`

Notes:

- clipboard state is validated with `ClipboardState`
- missing clipboard state defaults to empty text
- `clipboard.read` returns the current text in `FakeToolResult.data`
- `clipboard.write` replaces the clipboard text

Phase 1 Boundaries

These adapters are not real API clients. They should stay deterministic and side-effect-free outside their in-memory state.

Do not add OAuth, HTTP calls, browser automation, real clipboard access, or real Google/Notion API logic here. Real adapters should come only after fake adapters, the runner, and baseline scoring work end to end.

Testing

Each adapter has focused unit tests under `tests/unit/`:

- `test_fake_calendar.py`
- `test_fake_notion.py`
- `test_fake_gmail.py`
- `test_fake_clipboard.py`

Run all fake adapter tests with:

```
.venv\Scripts\python -m pytest tests\unit\test_fake_calendar.py tests\unit\test_fake_notion.py
tests\unit\test_fake_gmail.py tests\unit\test_fake_clipboard.py
```

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.